IoT Course — Hands-On #6

White Cube GUI Application — Assignment #3

Student: Yakira Siboni

File: white_cube_app.py

1. Overview

This report documents the design and implementation of a full PyQt5 GUI application for connecting to a White Cube IoT broker, subscribing to sensor data, handling events from a Reed sensor, and controlling a Relay device. The application was built using the provided template files (gui_main.py, gui_helpers.py, IoT_MQ_main.py, example_connect.py) as a foundation.

The application is implemented in a single file (white_cube_app.py) consisting of 499 lines of Python. It uses paho-mqtt v2 with thread-safe Qt signal bridging to safely update the GUI from MQTT callbacks running on a background thread.

2. Application Architecture

The application is structured as a QMainWindow containing five dockable panels (QDockWidget). Each panel handles a specific area of functionality:

Class / Component	Type	Description
MqttSignals	QObject	Thread-safe Qt signals bridging paho's background thread to the Qt main thread
Mqtt_client	MQTT wrapper	Wraps paho-mqtt: connect, disconnect, subscribe, publish, relay_on/off
MainDock	QDockWidget	Connection panel: Host, Port, Client ID, Username, Password, Keep Alive, Run Time, SSL, Clean Session, Connect button, status label
PublishDock	QDockWidget	Publish panel: Topic, QOS, Retain checkbox, Message box, Publish button
SubscribeDock	QDockWidget	Subscribe panel: Topic, QOS, Received messages area, Subscribe/Unsubscribe button
EventHandlerDock	QDockWidget	Reed sensor event handler: Device ID filter, Enable/Sound alarm checkboxes, State display, timestamped Alert Log
DeviceControlDock	QDockWidget	Relay device control: Topic Prefix, Device ID, Relay ON (green) and Relay OFF (red) buttons
MainWindow	QMainWindow	Main window — assembles all docks, connects Qt signals to message handler

2.1 Thread Safety — MqttSignals Bridge

A key design decision was the use of a QObject signal bridge (MqttSignals) to handle the thread boundary between paho-mqtt and Qt. Paho fires callbacks on its own background thread — directly updating Qt widgets from there causes crashes. The solution emits Qt signals from the paho callbacks, which are automatically queued to the main thread by Qt's event loop:

- on_connect → signals.connected.emit()
- on_disconnect → signals.disconnected.emit()
- on_message → signals.message_received.emit(topic, payload)

3. Application Demo

The following screenshots were extracted from the application demonstration recording, showing the app progressing through its key states.

3.1 Initial State — Disconnected

On launch the app shows all connection fields pre-filled with defaults. The Connect button is red and the Status label shows "Disconnected". All panels are already visible and ready.

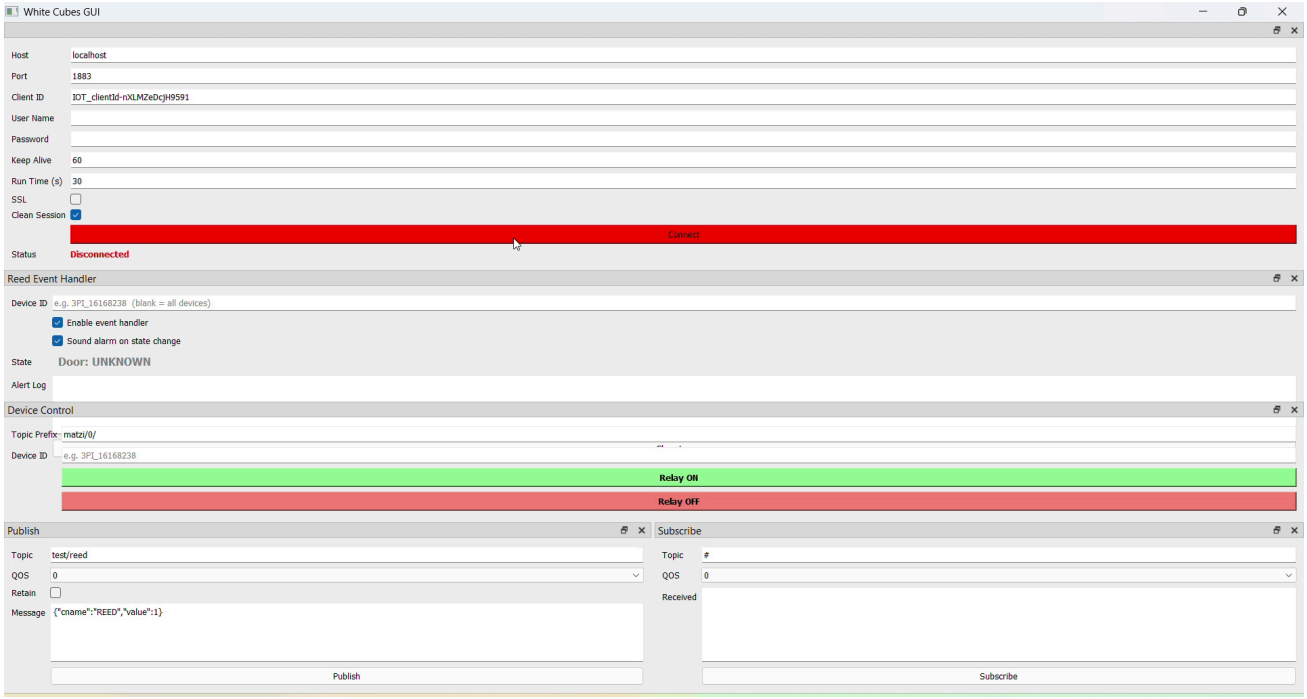

Figure 1: App launch — disconnected state, all panels visible

3.2 Connected — First Reed Event

After clicking Connect, the button turns green and the Status label shows "Connected — 24s". The Reed Event Handler panel shows "Door: OPEN" in red, with the Alert Log recording the first timestamped event: [16:32:13] test/reed → OPEN. The Subscribe panel displays the raw message payload received.

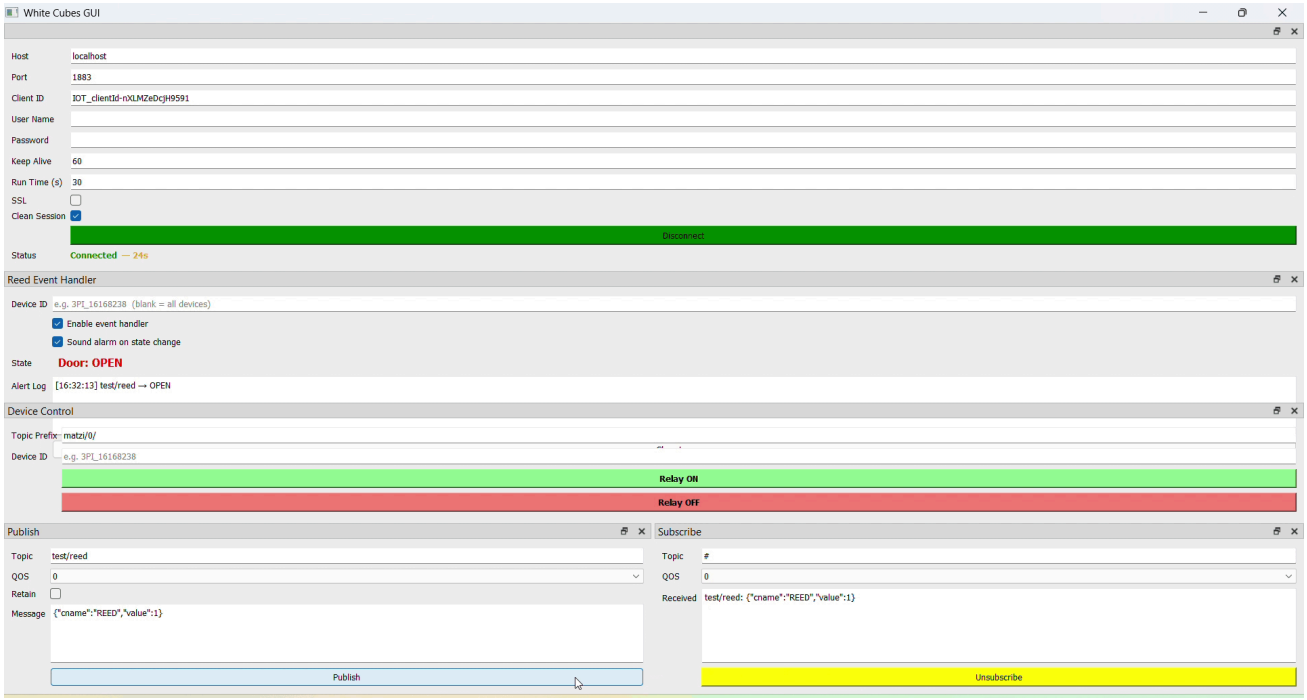

Figure 2: Connected — Reed sensor fires OPEN event, alert log updates

3.3 Multiple Events — Alert Log Building

After further sensor activity, the Alert Log accumulates multiple timestamped entries: OPEN at 16:32:13 and CLOSED at 16:32:20. The Subscribe panel shows all raw messages received. The countdown shows "Connected — 11s".

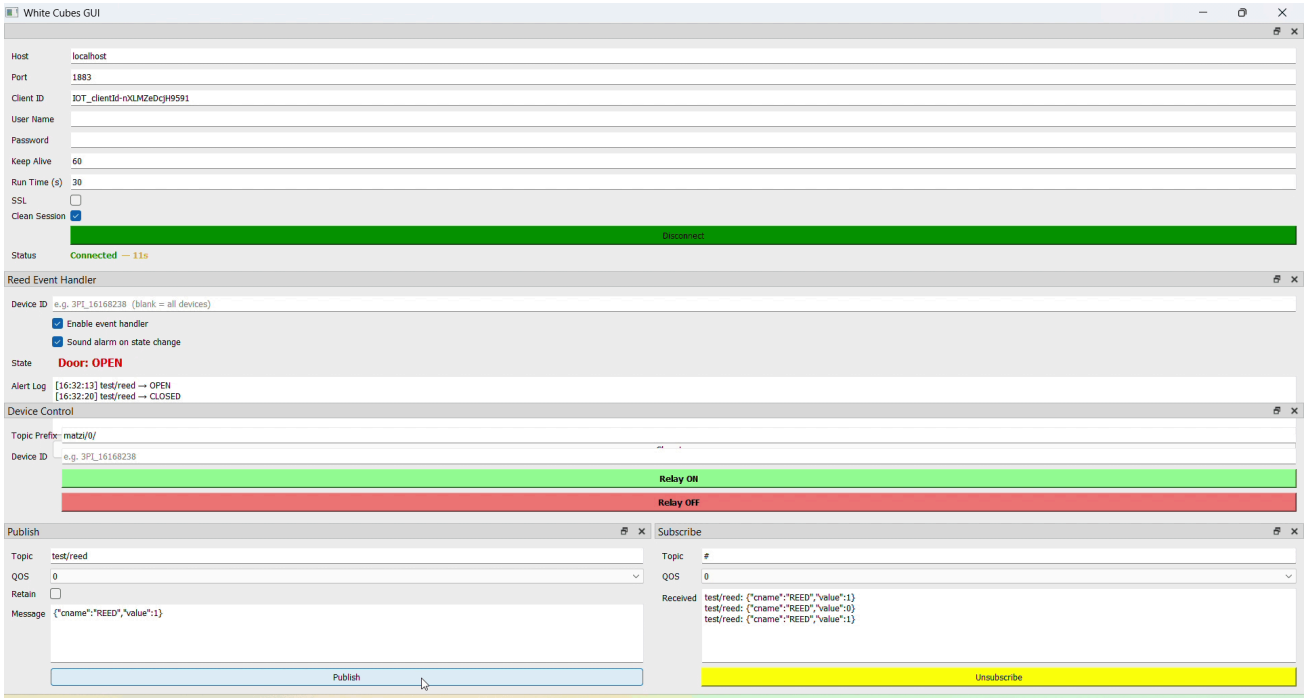

Figure 3: Alert log with OPEN and CLOSED events, multiple messages in Subscribe panel

3.4 Session Ended — Auto-Disconnect After Timeout

After the Run Time (30s) expires, the client automatically disconnects. The Connect button returns to red and the Status label shows "Disconnected". All recorded events remain preserved in the Alert Log and Subscribe panel — the session history is not cleared on disconnect.

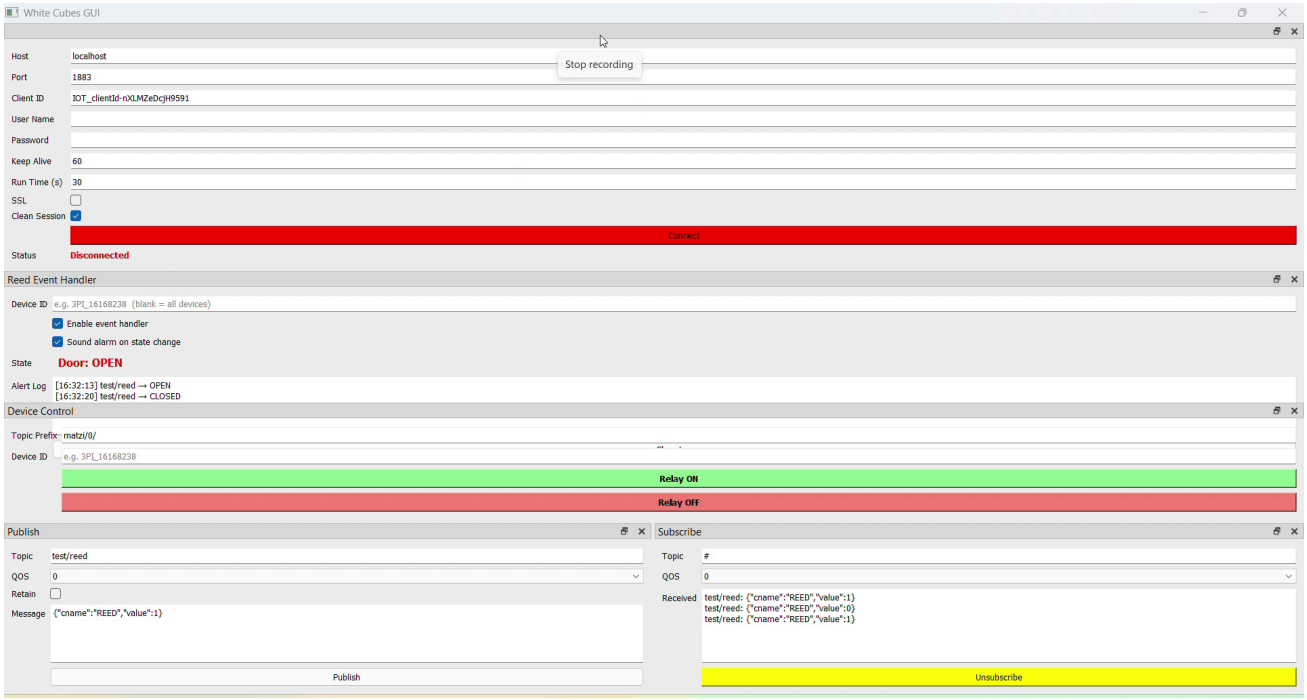

Figure 4: Post-timeout — auto-disconnected, full event history preserved

4. Key Implementation Details

4.1 Reed Event Handler

The EventHandlerDock filters incoming messages for Reed sensor events via check_reed_event(). When a matching message arrives:

- The payload JSON is parsed to extract the "value" field (1 = OPEN, 0 = CLOSED)
- The State label is updated with "Door: OPEN" or "Door: CLOSED" in colored text
- A timestamped entry is appended to the Alert Log
- If "Sound alarm on state change" is checked, winsound.Beep() is called
- A Device ID filter allows targeting a specific cube or all devices (blank = all)

4.2 Device Control — Relay

The DeviceControlDock provides two large buttons (Relay ON in green, Relay OFF in red) that call mc.relay_on() with the configured Topic Prefix and Device ID. The relay command JSON format is:

ON: {"type":"set_state", "action":"set_value", "addr":0, "cname":"ONOFF", "value":1}

OFF: {"type":"set_state", "action":"set_value", "addr":0, "cname":"ONOFF", "value":0}

5. Summary

The White Cube GUI application fully implements all requirements from Hands-On #6:

- Full MQTT connection management with configurable parameters and auto-timeout
- Continuous subscriber session with live message display
- Reed sensor event handler with sound alarm and timestamped alert log
- Relay device control with direct ON/OFF buttons
- Thread-safe architecture using Qt signal bridging for paho callbacks

The application was developed by extending and integrating the course-provided template files into a unified, working solution.